Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.1 sur 153 | difficulté 1 / 10

0.1. Afficher la lettre A

Une seule lettre, dans la toute première case de l'écran.

1. Ce qu'il faut comprendre

Les lettres sont **déjà dessinées** dans le programme : c'est la police, chargée au démarrage. Il suffit de dire laquelle afficher, et où.

`texte(0, 0, "A")` écrit **A** dans la case de la colonne **0**, ligne **0** : celle en haut à gauche. L'écran fait 20 colonnes (0 à 19) sur 18 lignes (0 à 17).

La ligne est écrite **avant** la boucle : on n'écrit le A qu'une fois, et il reste affiché.

La première ligne, `#include <texte>`, dit à la console : « ce programme se sert de `texte()` ». Aucune fonction de la console n'est là d'office : chacune prend de la **place dans la cartouche** (son code, ses lettres), et ne s'emploie que si on l'**inclut**, par son nom. Sans cette ligne, le compilateur refuse `texte()`, et dit quelle ligne écrire.

Au 0.0, il n'y avait aucun `#include` : `image()` ne coûte rien, elle est toujours là. À partir d'ici, chaque programme commence par la liste de ce qu'il emploie — une ligne par fonction, avec en commentaire ce qu'elle fait.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <texte>` → 0.0.2. La fonction texte() — écrire un mot

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **chaîne de caractères (texte)** — Des lettres entre guillemets, "COMME CECI" : un texte pris tel quel. La police de la console n'a que des majuscules, sans accents.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **texte()** — Écrit des lettres à l'écran, sur la grille de 20 colonnes et 18 lignes : texte(colonne, ligne, "MOT").
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// #include <texte> : « ce programme se sert de texte() ».`	Un commentaire : une phrase pour celui qui lit (« #include <texte> : « ce programme se sert de texte() ». »). La console l'ignore.
`// Une fonction de la console prend de la place dans la cartouche :`	Un commentaire : une phrase pour celui qui lit (« Une fonction de la console prend de la place dans la cartouche : »). La console l'ignore.
`// elle ne s'emploie que si on l'inclut, par son nom. Sans cette ligne,`	Un commentaire : une phrase pour celui qui lit (« elle ne s'emploie que si on l'inclut, par son nom. Sans cette ligne, »). La console l'ignore.
`// le compilateur refuse texte() — et dit quelle ligne écrire.`	Un commentaire : une phrase pour celui qui lit (« le compilateur refuse texte() — et dit quelle ligne écrire. »). La console l'ignore.

`#include <texte>   // écrit un texte à l'écran`	Demande à la console « texte » (ce que c'est : écrit un texte à l'écran, à la taille qu'on veut). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « écrit un texte à l'écran ».)
`int main() {          // Le programme commence ici.`	Le début du programme : la console commence ici, par la première ligne qui suit. (Le commentaire le dit : « Le programme commence ici. ».)
`texte(0, 0, "A");   // colonne 0, ligne 0 : la case en haut à gauche`	Appel de la fonction texte de la console, avec les arguments 0, 0, "A". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé. (Le commentaire le dit : « colonne 0, ligne 0 : la case en haut à gauche ».)
`// texte(colonne, ligne, "LETTRES") écrit des lettres à l'écran.`	Un commentaire : une phrase pour celui qui lit (« texte(colonne, ligne, "LETTRES") écrit des lettres à l'écran. »). La console l'ignore.
`// L'écran fait 20 colonnes (0 à 19) sur 18 lignes (0 à 17).`	Un commentaire : une phrase pour celui qui lit (« L'écran fait 20 colonnes (0 à 19) sur 18 lignes (0 à 17). »). La console l'ignore.
`// Les lettres à écrire se mettent entre guillemets.`	Un commentaire : une phrase pour celui qui lit (« Les lettres à écrire se mettent entre guillemets. »). La console l'ignore.
`// Écrit AVANT la boucle : une seule fois, et le A reste affiché.`	Un commentaire : une phrase pour celui qui lit (« Écrit AVANT la boucle : une seule fois, et le A reste affiché. »). La console l'ignore.
`while (true) {      // La boucle de jeu : recommence pour toujours...`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). (Le commentaire le dit : « La boucle de jeu : recommence pour toujours… ».)
`image();          // ... attendre l'image suivante (60 par seconde).`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « … attendre l'image suivante (60 par seconde). ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **203 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  main()           18 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EcrireTexte  — attend de pouvoir écrire une case
  EcrireTexte      10 o  ← texte()  — écrit un texte, lettre par lettre
  CopierTuiles     31 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles           16 o  ← CopierTuiles  — 1 lettre sur 43
```

```
  "A"              1 o  ← texte()  — un texte gravé
Lettres gravées : A
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  texte() × 1    11 o sur place, dans main()  + EcrireTexte (10 o)
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// #include <texte> : « ce programme se sert de texte() ».
// Une fonction de la console prend de la place dans la cartouche :
// elle ne s'emploie que si on l'inclut, par son nom. Sans cette ligne,
// le compilateur refuse texte() — et dit quelle ligne écrire.
#include <texte>   // écrit un texte à l'écran
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

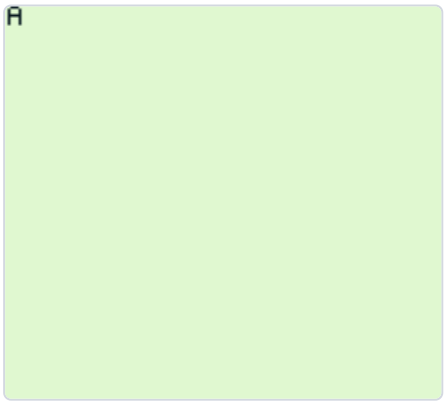
```
int main() {          // Le programme commence ici.
  texte(0, 0, "A");   // colonne 0, ligne 0 : la case en haut à gauche
                      // texte(colonne, ligne, "LETTRES") écrit des lettres à l'écran.
                      // L'écran fait 20 colonnes (0 à 19) sur 18 lignes (0 à 17).
                      // Les lettres à écrire se mettent entre guillemets.
                      // Écrit AVANT la boucle : une seule fois, et le A reste affiché.

  while (true) {      // La boucle de jeu : recommence pour toujours...
    image();          // ... attendre l'image suivante (60 par seconde).
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> Un A, tout en haut à gauche de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A est dans la case (0, 0)

✓ la case d'à côté est vide

Ensuite : **0.1.1. Afficher la lettre A — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++